

Informe Fullstack: Proyecto de Microservicios - Sistema de Gestión de “Banquetería Ecogarzones”

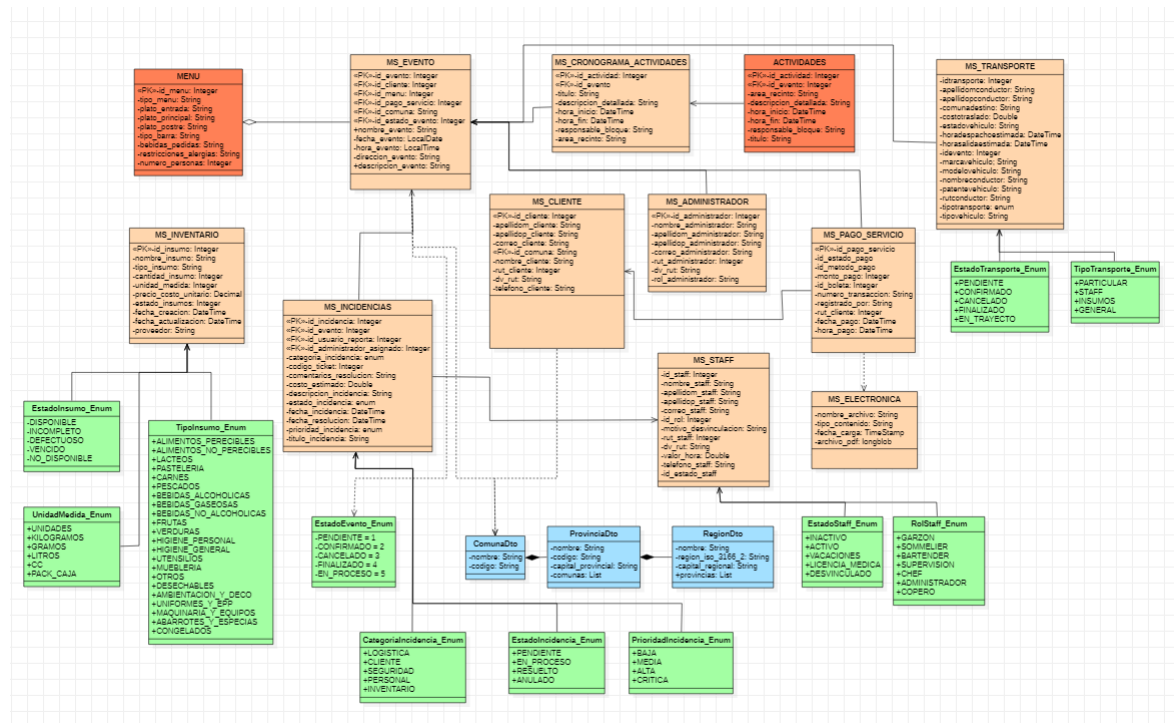
Estudiante: Matias Toloza Chaparro

Docente: Viviana Soledad Poblete Lopez
Asignatura: DESARROLLO FULLSTACK I_006D
Ingeniería en Informática - Duoc UC: Sede Maipú

La empresa Ecogarzones opera en el rubro gastronómico, especializándose en la prestación de servicios de atención para eventos sociales y corporativos de diversa envergadura, tales como bodas, aniversarios, cumpleaños, lanzamientos de marca y ceremonias formales. A pesar de contar con una trayectoria de cinco años de crecimiento sostenido en el mercado, la organización identificó la necesidad crítica de modernizar su infraestructura operativa y su sistema de gestión, principalmente en el agendamiento de servicios y en la administración de las fichas de los clientes que solicitan asistencia.

Para mitigar estas deficiencias y automatizar los procesos manuales, se diseñó e implementó GestionEcogarzones, una solución de software escalable y robusta fundamentada en el paradigma de microservicios distribuidos. Este ecosistema digital aborda y resuelve de forma aislada, descentralizada e interoperable los pilares fundamentales del dominio de la empresa, logrando la asignación eficiente y el control de asistencia en tiempo real del staff de garzones y bartenders, la gestión logística y el abastecimiento de insumos críticos de barra, y la auditoría automatizada de los flujos financieros derivados de cada contrato de banquetería.

Diagrama: Modelo de Base de Datos:



Como se puede ver en el diagrama, del sistema de GestionEcogarzones, el modelo de datos se estructura mediante una topología distribuida y desacoplada que clasifica sus componentes por color según su rol en el sistema. En primer lugar, se identifican diez microservicios principales (representados en color amarillo) que operan como raíces de agregación (*Aggregate Roots*) y delimitan los contextos de negocio independientes. En segundo lugar, el sistema incorpora once enumeradores (clases Enum en color verde), los cuales actúan como catálogos estáticos de validación en la capa de aplicación para restringir y tipar los estados y categorías del software. Por otra parte, se

integran tres objetos de transferencia de datos (clases DTO en color azul) dedicados exclusivamente a la estructuración de la jerarquía territorial y localización geográfica a través de la red. Finalmente, el diseño incluye tres entidades físicas de soporte local (representadas en color naranja), correspondientes a MENU y ACTIVIDADES, estructuras relacionales que operan supeditadas de forma estricta al ciclo de vida y base de datos de sus respectivos microservicios dominantes.

El servicio de autenticación, materializado en el ecosistema a través del componente MS_USUARIO (operando en el puerto 8081), actúa como el núcleo de seguridad perimetral y control de identidades de toda la arquitectura distribuida. Su propósito fundamental consiste en verificar la identidad de cualquier usuario (ya sean administradores de la empresa, clientes externos que requieran del servicio o personal del staff operativo de garzones, chefs, bartenders y demás) que intente interactuar con los recursos del sistema. Al centralizar este proceso, el microservicio garantiza que las credenciales de acceso se evalúen bajo un único estándar de seguridad y que, una vez comprobada la identidad, se le asigne al usuario un conjunto específico de privilegios o roles. Esta segmentación es crítica para resguardar la integridad de las operaciones de la empresa, asegurando que un garzón solo acceda a su cronograma o asistencia, mientras que un administrador posea el control de los flujos financieros, la gestión de inventario y la moderación total de la plataforma.

i. ¿Cómo se implementa la autenticación por token?

Para este ecosistema distribuido se ha implementado un mecanismo de autenticación basado en tokens, utilizando el estándar JSON Web Token (JWT). Este enfoque sigue una arquitectura totalmente desacoplada y sin estado (Stateless), eliminando la necesidad de almacenar sesiones en el servidor y permitiendo que la verificación de identidad sea altamente escalable. El flujo técnico de esta implementación opera bajo la siguiente secuencia de eventos:

- i. Primero, el cliente inicia el proceso enviando una solicitud HTTP POST al endpoint de autenticación, adjuntando en el cuerpo de la petición un objeto JSON con sus credenciales de acceso, compuestas por el nombre de usuario y la contraseña cifrada.
- ii. Segundo, el MS Autenticación intercepta la solicitud y valida los datos recibidos contrastándolos contra su propio esquema de persistencia relacional denominado usuario_ecogarzones. Si las credenciales coinciden de manera exacta, el servicio utiliza un algoritmo de cifrado criptográfico simétrico para generar una cadena de caracteres codificada llamada Token JWT. Este token incorpora internamente una serie de afirmaciones públicas firmadas, conocidas como claims, que contienen el identificador del usuario, su nombre y los roles asignados, además de una marca de tiempo que define su periodo estricto de expiración.
- iii. Tercero, el servidor responde al cliente de manera exitosa devolviendo este token en formato String dentro del cuerpo de la respuesta HTTP, junto con un código de estado 200 OK o 201 Created.
- iv. Cuarto, el cliente recibe el token y lo almacena localmente en la memoria de la aplicación o en el almacenamiento del navegador. Para cualquier petición posterior dirigida a los microservicios del negocio, el cliente debe adjuntar de manera obligatoria dicho token en la cabecera HTTP de la solicitud, utilizando el esquema de autorización estandarizado Authorization: Bearer seguido del token generado. Cuando la petición llega a los microservicios de destino, estos descodifican y validan la firma criptográfica del token de forma autónoma utilizando la misma clave secreta, otorgando el acceso de inmediato sin necesidad de realizar costosas consultas síncronas recurrentes a la base de datos de usuarios.

ii. Spring Security, Protección de Rutas y CORS

La infraestructura de resguardo técnico de los endpoints distribuidos se sostiene sobre tres pilares fundamentales provistos por el framework y la topología de la red:

Spring Security: Cada microservicio crítico y el API Gateway integran los componentes nativos de este framework para establecer un filtro de seguridad en serie, formalmente denominado `SecurityFilterChain`. Este componente actúa como un interceptor de bajo nivel en la pila de red que evalúa cada petición HTTP entrante antes de que llegue a la capa de los controladores REST. Spring Security se encarga de extraer la cabecera de autorización, validar la autenticidad del Token JWT adjunto y poblar el contexto de seguridad de la aplicación con la identidad y los roles del usuario firmante.

Protección de Rutas: A través de clases de configuración programáticas dentro del código Java, se establecen reglas estrictas para gobernar el acceso a las URIs del sistema. Se definen rutas públicas explícitas que prescinden de tokens de acceso para permitir flujos iniciales esenciales, como el registro de nuevos usuarios o el inicio de sesión. Por el contrario, los endpoints operacionales se configuran bajo esquemas de restricción basados en roles o autenticación obligatoria. Por ejemplo, se estructuran políticas que determinan que cualquier petición dirigida a la ruta del módulo de administración de alta jerarquía requiere obligatoriamente la presencia del rol administrativo del sistema mediante reglas estrictas de coincidencia de roles.

CORS (Cross-Origin Resource Sharing): Al tratarse de una arquitectura orientada a microservicios donde las aplicaciones cliente de Frontend y los servicios del Backend coexisten en puertos físicos o dominios totalmente independientes, el API Gateway centraliza de manera unificada la política global de intercambio de recursos de origen cruzado. La configuración de CORS en el Gateway intercepta las solicitudes de pre-vuelo emitidas por los navegadores y valida explícitamente el origen de la petición, permitiendo llamadas exclusivas procedentes de los dominios autorizados de desarrollo o producción del Frontend. Asimismo, inyecta dinámicamente en las respuestas de red las cabeceras HTTP de control de acceso requeridas, especificando los verbos permitidos, las cabeceras admitidas y los tiempos de persistencia de la configuración, solucionando de forma centralizada los bloqueos de seguridad en el cliente.

Cada uno de los componentes de la arquitectura del software ha sido estructurado de manera individual siguiendo los lineamientos de diseño del patrón `Controller-Service-Repository (CSR)`. Este enfoque de desarrollo promueve la alta cohesión del código, facilita el mantenimiento modular y disminuye sustancialmente el acoplamiento técnico entre las operaciones del negocio y la persistencia física de datos. En el diseño de este ecosistema, la organización del microservicio de Eventos se divide estrictamente en tres niveles funcionales:

- La capa **Controller**, representada por clases como `"EventoController.java"`, es la capa de presentación encargada exclusivamente de exponer los endpoints HTTP REST hacia el API Gateway. Su responsabilidad se limita a recibir las solicitudes del cliente, interceptar los parámetros de ruta, validar las estructuras de entrada y despachar las respuestas correspondientes junto a sus respectivos códigos de estado HTTP.
- La capa **Service**, representada como `"EventoService.java"`, constituye el núcleo operativo del dominio. En este nivel se implementa el cien por ciento de la lógica de negocio, los flujos transaccionales y las restricciones lógicas, permaneciendo completamente aislada de los protocolos de transporte web y de las tecnologías de bases de datos.

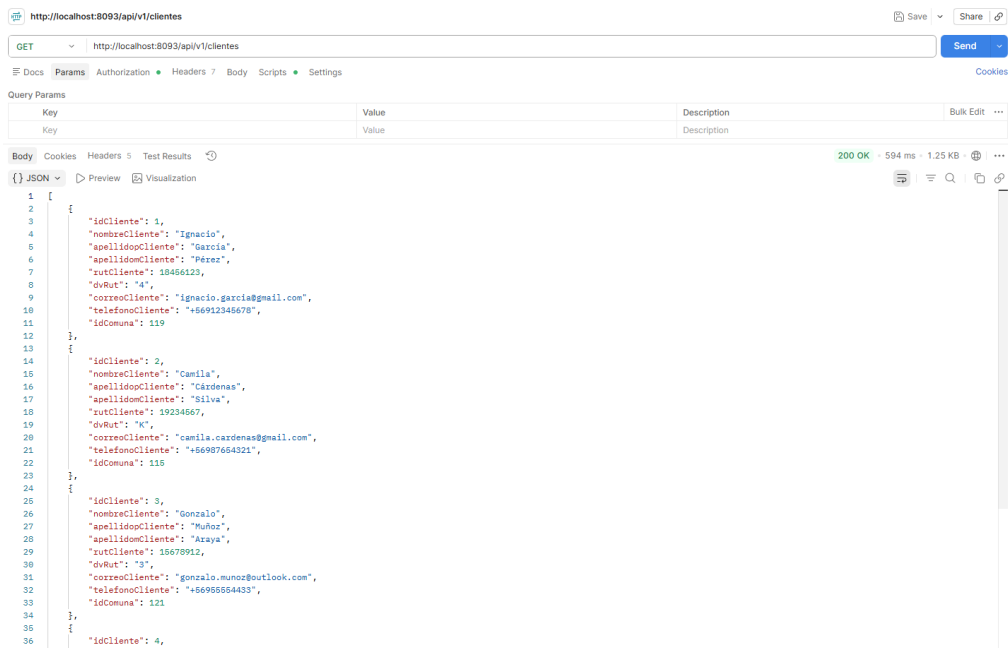
- Por último, la capa **Repository** (“EventoRepository.java”), gestiona la abstracción de datos interactuando con el motor relacional de MySQL mediante Spring Data JPA. Administra de forma nativa la persistencia orientada a objetos de la entidad Evento, suprimiendo la codificación de sentencias SQL manuales.

Para ejemplificar el comportamiento y alcance de la capa de presentación en esta arquitectura, se presenta el código fuente correspondiente al controlador del microservicio de Clientes (ClienteController). En este fragmento se detalla la implementación de la anotación `@RestController` y los mapeos de rutas basados en los verbos HTTP estándar mediante las directivas `@GetMapping`, `@PostMapping`, `@PutMapping` y `@DeleteMapping`. Estas operaciones componen el CRUD elemental necesario para la gestión y persistencia de las fichas de clientes en el sistema de banquetería. Al realizar una petición de prueba mediante el método GET en herramientas como Postman o Thunder Client hacia la URL base (/api/v1/clientes), el controlador intercepta la solicitud, consulta el repositorio y retorna con éxito la colección completa de registros en formato JSON.

```
Clientes > src > main > java > com > ecogarzones > clientes > controller > ClienteController.java > Language Support for Java(TM) by Red Hat > ClienteController

17
18 ~ ClienteService ClienteRepository
19 @RestController
20 @RequestMapping("/api/v1/clientes")
21 public class ClienteController {
22
23     ~ ClienteRepository
24     @Autowired
25     private ClienteRepository clienteRepository;
26
27     ~ ClienteService
28     ClienteController(ClienteService clienteService) {
29     }
30
31     http://127.0.0.1:8093/api/v1/clientes
32     @GetMapping
33     public ResponseEntity<List<Cliente>> listar(){
34         List<Cliente> clientes = clienteRepository.findAll();
35         if (clientes.isEmpty()){
36             return ResponseEntity.noContent().build();
37         }
38         return ResponseEntity.ok(clientes);
39     }
40
41     http://127.0.0.1:8093/api/v1/clientes/{id}
42     @GetMapping("/{id}")
43     public ResponseEntity<Cliente> buscarCliente(@PathVariable Integer id){
44         try{
45             Cliente cliente = clienteRepository.findById(id).orElse(null);
46             if (cliente == null) {
47                 return ResponseEntity.notFound().build();
48             }
49             return ResponseEntity.ok(cliente);
50         } catch (Exception e){
51             return ResponseEntity.notFound().build();
52         }
53     }
54
55     http://127.0.0.1:8093/api/v1/clientes
56     @PostMapping
57     public ResponseEntity<Cliente> crear(@RequestBody Cliente cliente){
58         if (clienteRepository.existsByRutCliente(cliente.getRutCliente())) {
59             return ResponseEntity.badRequest().build();
60         }
61         if (clienteRepository.existsByCorreoCliente(cliente.getCorreoCliente())) {
62             return ResponseEntity.badRequest().build();
63         }
64     }
65 }
```

- Uso de GET para obtener registros totales/ id particular:



http://localhost:8093/api/v1/clientes

GET http://localhost:8093/api/v1/clientes

Params Authorization Headers 7 Body Scripts Settings

Query Params

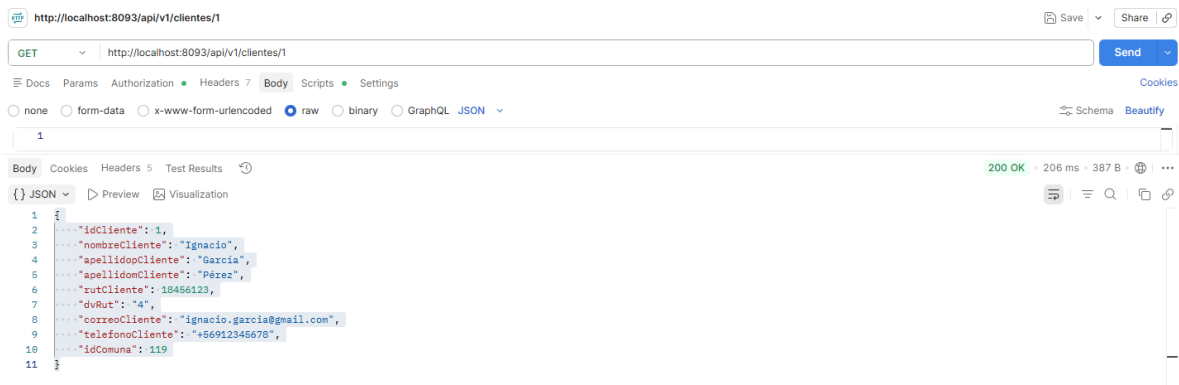
Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 5 Test Results

200 OK - 594 ms - 1.25 KB

```
{
  "idCliente": 1,
  "nombreCliente": "Ignacio",
  "apellidoopCliente": "Garcia",
  "apellidoomCliente": "Perez",
  "rutCliente": "18456123",
  "dvRut": "4",
  "correoCliente": "ignacio.garcia@gmail.com",
  "telefonoCliente": "+56912345678",
  "idComuna": 119
},
{
  "idCliente": 2,
  "nombreCliente": "Camila",
  "apellidoopCliente": "Cardenas",
  "apellidoomCliente": "Silva",
  "rutCliente": "19234567",
  "dvRut": "K",
  "correoCliente": "camila.cardenas@gmail.com",
  "telefonoCliente": "+56997664321",
  "idComuna": 115
},
{
  "idCliente": 3,
  "nombreCliente": "Gonzalo",
  "apellidoopCliente": "Munoz",
  "apellidoomCliente": "Araya",
  "rutCliente": "15678912",
  "dvRut": "3",
  "correoCliente": "gonzalo.munoz@outlook.com",
  "telefonoCliente": "+56955554433",
  "idComuna": 121
},
{
  "idCliente": 4,
```

- Uso de PUT para Actualizar Registro:



http://localhost:8093/api/v1/clientes/1

GET http://localhost:8093/api/v1/clientes/1

Params Authorization Headers 7 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

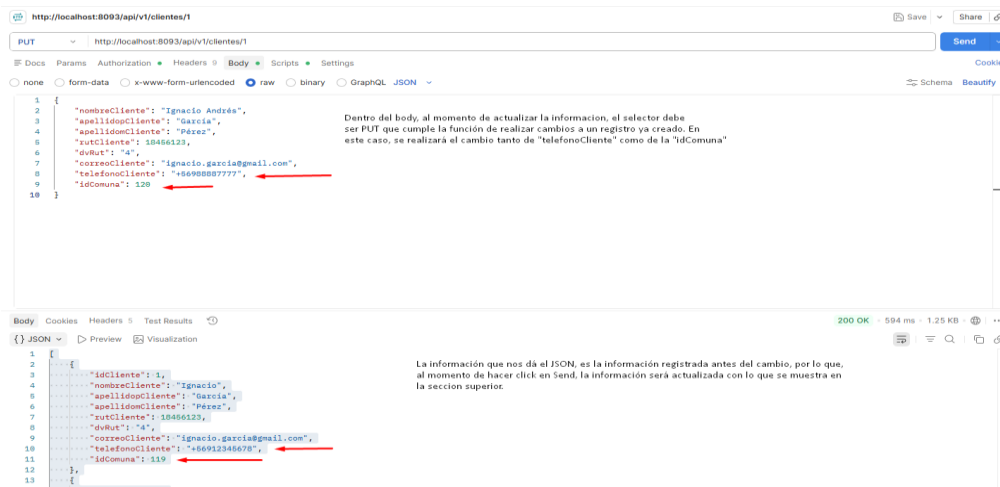
Schema Beautify

1

Body Cookies Headers 5 Test Results

200 OK - 206 ms - 387 B

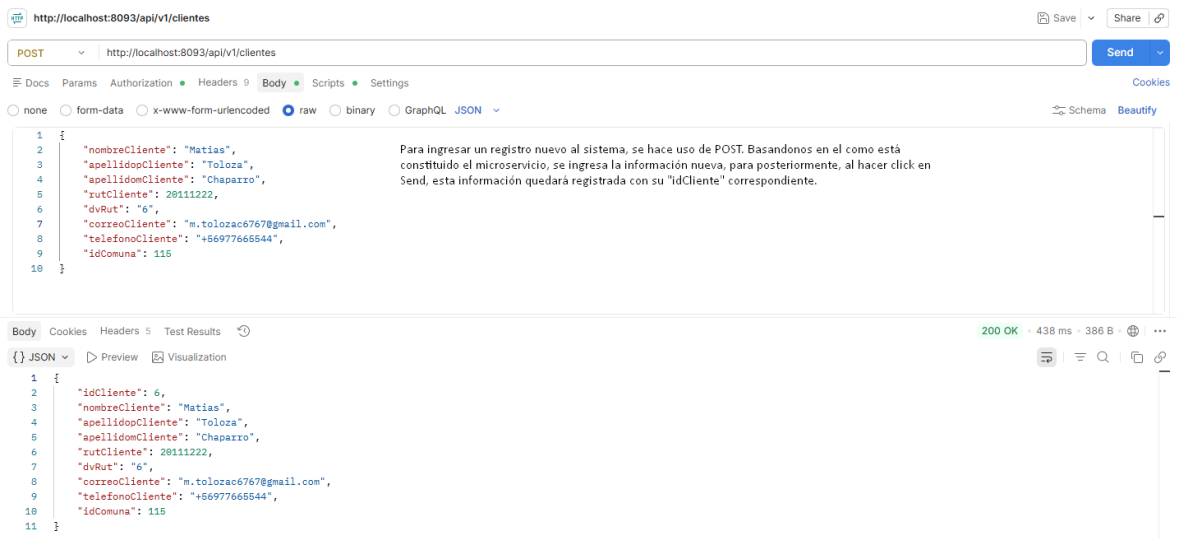
```
{
  "idCliente": 1,
  "nombreCliente": "Ignacio",
  "apellidoopCliente": "Garcia",
  "apellidoomCliente": "Perez",
  "rutCliente": "18456123",
  "dvRut": "4",
  "correoCliente": "ignacio.garcia@gmail.com",
  "telefonoCliente": "+56912345678",
  "idComuna": 119
}
```



- Uso del DELETE para borrar registro:



- Uso de POST para crear un registro:



Comunicación entre microservicios: ¿cómo se implementa?

En esta arquitectura distribuida, la comunicación entre los diferentes módulos se realiza a través de la red mediante dos estrategias principales, asegurando la autonomía y eficiencia del sistema: Por un lado, la comunicación sincronica se utiliza cuando un servicio necesita información inmediata de otro para continuar su proceso (por ejemplo, el MS_Evento consultando datos al MS_Cliente). Se implementa mediante el protocolo *HTTP/REST* enviando datos en formato JSON. En Spring Boot se facilita mediante OpenFeign, un cliente declarativo que permite invocar endpoints de otros microservicios de manera directa mediante interfaces simples.

Por otro lado, la comunación asincronica se utiliza en procesos que no requieren respuesta inmediata para evitar el acoplamiento y los fallos en cadena (por ejemplo, el MS_Pago notificando al MS_Electrónica para generar una boleta). Se implementa utilizando un Message Broker (como RabbitMQ o Apache Kafka). El servicio emisor publica un evento en el broker y continúa su ejecución de inmediato, mientras que los servicios interesados consumen el mensaje y reaccionan en segundo plano, garantizando la alta disponibilidad del sistema.

Swagger: ¿Qué es y en qué consiste?

El Swagger consiste en un conjunto de herramientas de código abierto construido en torno a la especificación OpenAPI (OAS) que se utiliza para diseñar, construir, documentar y consumir servicios web RESTful. Básicamente, consiste en automatizar el proceso de documentación de una API. En lugar de escribir un documento manual en Word o PDF que se desactualiza cada vez que modificas el código, el Swagger "lee" tu código fuente (controladores, rutas, modelos) y genera automáticamente una especificación técnica e incluso una interfaz gráfica (Swagger UI).

Ahora, para poder integrar el Swagger a un proyecto de microservicio, como el de GestionEcogarzones, dentro del archivo "pom.xml", se debe añadir el artefacto "springdoc-openapi-starter-webmvc-ui". Esto permite que el framework analice los controladores y genere de forma automática la especificación OpenAPI. Cabe mencionar que esto se debe replicar con todos los archivos "pom.xml" en el resto de microservicios. Al mismo tiempo, en el archivo "application.properties", se definen parámetros básicos como el título de la API, la versión y la ruta de acceso personalizada para la interfaz gráfica (por ejemplo, /swagger-ui.html). Finalmente, Se enriquecen las clases controladoras, que fueron explicadas previamente, con anotaciones de OpenAPI (como @Tag y @Operation). Estas etiquetas permiten añadir resúmenes, descripciones y respuestas esperadas a cada endpoint (GET, POST, PUT, DELETE).

Para ejemplificar, como parte del proceso de documentación técnica automatizada para el ecosistema *GestionEcogarzones*, se implementó la dependencia *Springdoc OpenAPI* en el microservicio de Clientes (ejecutándose en el puerto 8093). Al acceder a la ruta de inspección en el entorno local, se despliega la interfaz gráfica de Swagger UI (ver *Figura X*), la cual expone formalmente la especificación de la API basada en el estándar OpenAPI 3.1.

Fragmento de cliente-controller:

```
17
18  - ClienteService ClienteRepository
19  @RestController
20  @RequestMapping("/api/v1/clientes")
21  public class ClienteController {
22
23      - ClienteRepository
24      @Autowired
25      private ClienteRepository clienteRepository;
26
27      - ClienteService
28      ClienteController(ClienteService clienteService) {
29      }
30
31      http://127.0.0.1:8093/api/v1/clientes (Count=1 Total=0,42s Max=0,00s)
32      @GetMapping
33      public ResponseEntity<List<Cliente>> listar(){
34          List<Cliente> clientes = clienteRepository.findAll();
35          if (clientes.isEmpty()){
36              return ResponseEntity.noContent().build();
37          }
38          return ResponseEntity.ok(clientes);
39      }
40
41      http://127.0.0.1:8093/api/v1/clientes/{id} (Count=3 Total=0,21s Max=0,00s)
42      @GetMapping("/{id}")
43      public ResponseEntity<Cliente> buscarCliente(@PathVariable Integer id){
44          try{
45              Cliente cliente = clienteRepository.findById(id).orElse(other: null);
46              if (cliente == null) {
47                  return ResponseEntity.notFound().build();
48              }
49              return ResponseEntity.ok(cliente);
50          } catch (Exception e){
51              return ResponseEntity.notFound().build();
52          }
53      }
54  }
```

Panel de control interactivo de Swagger UI para cliente-controller:

The screenshot shows the Swagger UI interface for the 'cliente-controller' API. The browser address bar shows 'localhost:8093/swagger-ui/index.html/#/'. The Swagger logo is visible, along with a search bar containing '/v3/api-docs' and an 'Explore' button. Below the logo, it says 'OpenAPI definition' with a version 'v0 OAS 3.1' and a link to '/v3/api-docs'. A 'Servers' section shows a dropdown menu with 'http://localhost:8093 - Generated server url'. The main section, titled 'cliente-controller', lists several API endpoints with their methods and paths: GET /api/v1/clientes/{id}, PUT /api/v1/clientes/{id}, DELETE /api/v1/clientes/{id}, GET /api/v1/clientes, and POST /api/v1/clientes. Each endpoint has a dropdown arrow next to it. At the bottom, there is a 'Schemas' section with a dropdown arrow, and a 'Cliente' schema is partially visible with an 'Expand all object' link.

Fragmento de incidencia-controller:

```
@RestController
@RequestMapping("/api/v1/incidencias")
public class ReporteIncidenciaController {

    @Autowired
    private ReporteIncidenciaService service;

    @GetMapping
    public List<ReporteIncidencia> listarTodas() {
        return service.obtenerTodos();
    }

    @PostMapping
    public ResponseEntity<?> crearIncidencia(@RequestBody ReporteIncidencia reporte) {
        try {
            ReporteIncidencia nuevoReporte = service.guardar(reporte);
            return ResponseEntity.ok(nuevoReporte);
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(Map.of(k1: "status", v1: "error", k2: "error", e.getMessage()));
        } catch (IllegalStateException e) {
            if (e.getMessage().contains("disponible")) {
                return ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(Map.of(k1: "status", v1: "failed", k2: "error", e.getMessage()));
            }
            return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(Map.of(k1: "status", v1: "error", k2: "error", e.getMessage()));
        }
    }

    @GetMapping("/ticket/{codigo}")
    public ResponseEntity<ReporteIncidencia> buscarPorTicket(@PathVariable String codigo) {
        return service.obtenerPorTicket(codigo)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Panel de control interactivo de Swagger UI para incidencia-controller

Servers
http://localhost:8083 - Generated server url

reporte-incidencia-controller

PUT	/api/v1/incidencias/{id}/resolver	▼
GET	/api/v1/incidencias	▼
POST	/api/v1/incidencias	▼
GET	/api/v1/incidencias/usuario/{idUser}/alerta-sobrecarga	▼
GET	/api/v1/incidencias/ticket/{codigo}	▼
GET	/api/v1/incidencias/filtro/prioridad	▼
GET	/api/v1/incidencias/filtro/estado	▼
GET	/api/v1/incidencias/filtro/avanzado	▼
GET	/api/v1/incidencias/costo-financiero	▼
GET	/api/v1/incidencias/configuracion-sla	▼
GET	/api/v1/incidencias/administrador/{idAdmin}	▼

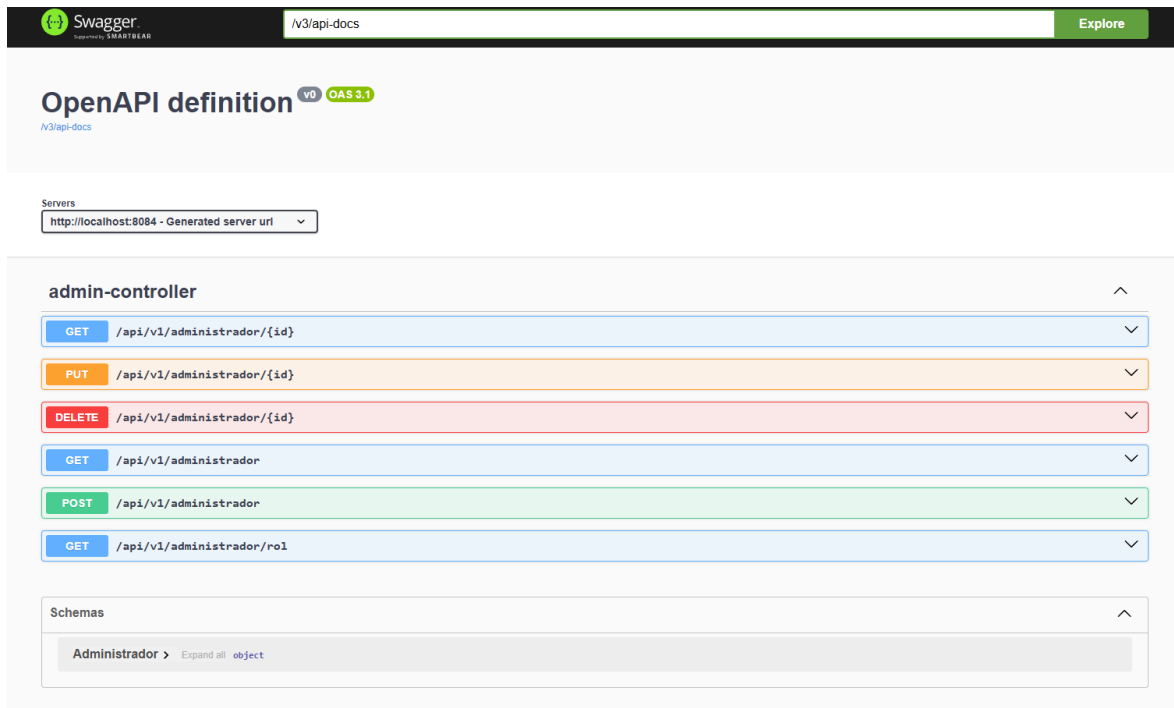
Schemas

ReporteIncidencia >

Fragmento de admin-controller:

```
20 @RestController
21 @RequestMapping("api/v1/administrador")
22 public class AdminController {
23
24     @Autowired
25     private AdminRepository adminRepository;
26
27     AdminController(AdminService adminService) {
28     }
29
30     @GetMapping
31     public ResponseEntity<List<Administrador>> listar(){
32         List<Administrador> administradores = adminRepository.findAll();
33         if (administradores.isEmpty()){
34             return ResponseEntity.noContent().build();
35         }
36         return ResponseEntity.ok(administradores);
37     }
38
39
40     @GetMapping("/{id}")
41     public ResponseEntity<Administrador> buscarAdministrador(@PathVariable Integer id){
42         try{
43             Administrador administrador = adminRepository.findById(id).orElse(null);
44             if (administrador == null) {
45                 return ResponseEntity.notFound().build();
46             }
47             return ResponseEntity.ok(administrador);
48         }
49         catch(Exception e){
50             return ResponseEntity.notFound().build();
51         }
52     }
53 }
```

Panel de control interactivo de Swagger UI para admin-controller:



Testing: ¿Qué es y en qué consiste?

El Testing (o pruebas de software) es un proceso disciplinado y sistemático cuyo objetivo es evaluar, verificar y validar que un sistema de software funcione exactamente como se especificó en los requisitos de negocio. Consiste en ejecutar el código (o partes del mismo) bajo condiciones controladas para comparar el comportamiento real del programa contra el comportamiento esperado, identificando errores (*bugs*), fallos de lógica o vulnerabilidades de seguridad antes de que el producto llegue al usuario final.

Ahora, en una arquitectura de microservicios, como el sistema de *GestionEcogarzones*, el testing se divide en una pirámide de pruebas que abarca desde la verificación de una sola función en aislamiento, pasando por la integración de módulos, hasta la comunicación completa entre servicios. Así mismo, para mantener las pruebas ordenadas, legibles y estandarizadas a nivel profesional, se utiliza universalmente el patrón AAA. Este patrón divide el cuerpo de cada prueba unitaria en tres etapas secuenciales claras:

- i. **Arrange (Organizar / Preparar):** En esta primera fase se configura el escenario inicial para la prueba. Aquí se definen e inician las variables de entrada, se crean los objetos de prueba (datos simulados o fixtures) y se establece el comportamiento esperado de los componentes simulados (Mocks).
- ii. **Act (Actuar / Ejecutar):** Es el momento donde se ejecuta la acción principal de la prueba; es decir, se invoca de manera concreta al método o función que se desea testear, pasándole los datos preparados en la fase de Arrange. Normalmente consiste en una sola línea de código que captura el resultado o la respuesta del sistema.

- iii. **Assert (Afirmar / Verificar):** Es la etapa de validación final. Aquí se contrasta el resultado obtenido en la fase de Act con el valor que teóricamente esperábamos recibir. Si la afirmación se cumple, la prueba pasa con éxito; si los valores difieren, el test falla y te indica exactamente qué falló.

Para ejemplificar con uno de los microservicios de GestionEcogarzones, tenemos el caso MS_ADMIN. Dentro de dicho microservicio se desarrolló la suite de pruebas unitarias AdminServiceTest. Esta clase utiliza JUnit 5 como motor de pruebas y Mockito como framework de simulación (*mocking*). Su objetivo principal es verificar de forma aislada la lógica de negocio alojada en AdminService, anulando cualquier dependencia física o comunicación con la base de datos real mediante el uso de repositorios simulados (@Mock).

```
@Test
@DisplayName("GIVEN admin existente WHEN crear THEN debe lanzar IllegalArgumentException")
void testCrearAdministradorDuplicado() {
    // Arrange
    Administrador admin = new Administrador();
    admin.setRutAdministrador(rutAdministrador: 12345678);
    when(adminRepository.existsByRutAdministrador(rutAdministrador: 12345678)).thenReturn(true);

    // Act & Assert
    assertThrows(IllegalArgumentException.class, () -> adminService.crearAdministrador(admin));
}

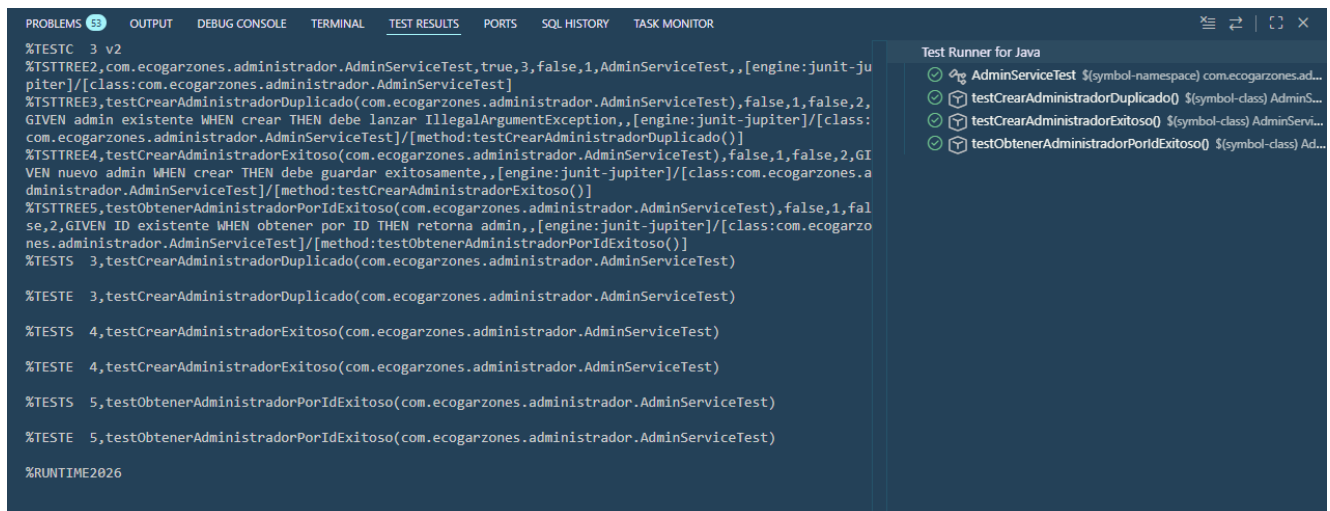
@Test
@DisplayName("GIVEN ID existente WHEN obtener por ID THEN retorna admin")
void testObtenerAdministradorPorIdExitoso() {
    // Arrange
    Administrador admin = new Administrador();
    when(adminRepository.findById(id: 1)).thenReturn(Optional.of(admin));

    // Act
    Administrador resultado = adminService.obtenerAdministradorPorId(idAdministrador: 1);

    // Assert
    assertNotNull(resultado);
    verify(adminRepository, times(1)).findById(id: 1);
}

17
18 @ExtendWith(MockitoExtension.class)
19 public class AdminServiceTest {
20
21     @Mock
22     private AdminRepository adminRepository;
23
24     @InjectMocks
25     private AdminService adminService;
26
27     @Test
28     @DisplayName("GIVEN nuevo admin WHEN crear THEN debe guardar exitosamente")
29     void testCrearAdministradorExitoso() {
30         // Arrange
31         Administrador admin = new Administrador();
32         admin.setRutAdministrador(rutAdministrador: 12345678);
33         admin.setCorreoAdministrador(correoAdministrador: "test@test.com");
34
35         when(adminRepository.existsByRutAdministrador(anyInt())).thenReturn(false);
36         when(adminRepository.existsByCorreoAdministrador(anyString())).thenReturn(false);
37         when(adminRepository.save(any(Administrador.class))).thenReturn(admin);
38
39         // Act
40         Administrador resultado = adminService.crearAdministrador(admin);
41
42         // Assert
43         assertNotNull(resultado);
44         verify(adminRepository, times(1)).save(admin);
45     }
46 }
```

Resultado de Testing en MS_ADMIN:



PROBLEMS 53 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS SQL HISTORY TASK MONITOR

%TESTC 3 v2
%TSTTREE2,com.ecogarzones.administrador.AdminServiceTest,true,3,false,1,AdminServiceTest,,[engine:junit-jupiter]/[class:com.ecogarzones.administrador.AdminServiceTest]
%TSTTREE3,testCrearAdministradorDuplicado(com.ecogarzones.administrador.AdminServiceTest),false,1,false,2,GIVEN admin existente WHEN crear THEN debe lanzar IllegalArgumentException,,[engine:junit-jupiter]/[class:com.ecogarzones.administrador.AdminServiceTest]/[method:testCrearAdministradorDuplicado()
%TSTTREE4,testCrearAdministradorExitoso(com.ecogarzones.administrador.AdminServiceTest),false,1,false,2,GIVEN nuevo admin WHEN crear THEN debe guardar exitosamente,,[engine:junit-jupiter]/[class:com.ecogarzones.administrador.AdminServiceTest]/[method:testCrearAdministradorExitoso()
%TSTTREE5,testObtenerAdministradorPorIdExitoso(com.ecogarzones.administrador.AdminServiceTest),false,1,false,2,GIVEN ID existente WHEN obtener por ID THEN retorna admin,,[engine:junit-jupiter]/[class:com.ecogarzones.administrador.AdminServiceTest]/[method:testObtenerAdministradorPorIdExitoso()
%TESTS 3,testCrearAdministradorDuplicado(com.ecogarzones.administrador.AdminServiceTest)

%TESTE 3,testCrearAdministradorDuplicado(com.ecogarzones.administrador.AdminServiceTest)

%TESTS 4,testCrearAdministradorExitoso(com.ecogarzones.administrador.AdminServiceTest)

%TESTE 4,testCrearAdministradorExitoso(com.ecogarzones.administrador.AdminServiceTest)

%TESTS 5,testObtenerAdministradorPorIdExitoso(com.ecogarzones.administrador.AdminServiceTest)

%TESTE 5,testObtenerAdministradorPorIdExitoso(com.ecogarzones.administrador.AdminServiceTest)

%RUNTIME2026

Test Runner for Java

- ✓ AdminServiceTest \${symbol-namespace} com.ecogarzones.ad...
- ✓ testCrearAdministradorDuplicado() \${symbol-class} AdminS...
- ✓ testCrearAdministradorExitoso() \${symbol-class} AdminServ...
- ✓ testObtenerAdministradorPorIdExitoso() \${symbol-class} Ad...

Como se ven en las imágenes, en la pestaña de “Test Results” se valida formalmente la ejecución exitosa de las pruebas unitarias del componente *AdminServiceTest*. La evidencia visual confirma una aprobación completa, donde el Test Runner para Java muestra iconos verdes con verificación,

indicando que la suite se ejecutó sin fallos ni errores críticos. Asimismo, JUnit 5 validó correctamente los tres flujos de negocio definidos: la detección de administradores duplicados mediante la excepción esperada, la creación exitosa de un nuevo administrador bajo condiciones normales y la correcta recuperación de un registro mediante su identificador único. Finalmente, los logs de ejecución reflejan de manera consistente las marcas de inicio y término exitoso de cada prueba, lo que respalda la correcta trazabilidad y solidez de la lógica implementada antes de su despliegue.